RAG keeps the source material outside the model. Your application indexes documents, searches them when a question arrives, retrieves useful passages and supplies those passages as context. The model writes an answer grounded in that material.

Modern retrieval can go beyond a single vector search. Qdrant, for example, documents hybrid retrieval combining dense semantic search with sparse lexical search, with reranking used to improve precision. We will build this properly later; for now, remember the principle: when information changes, retrieve it rather than trying to bake it permanently into the model.

Fine-tuning: change the model's behaviour

Fine-tuning adjusts model weights using examples. It becomes useful when the problem is not merely "the model lacks a fact" but "the model repeatedly behaves in the wrong way".

Perhaps you need a compact model to transform messy support tickets into a specific classification format or consistently follow a domain-specific conversational pattern. A good training set can make that behaviour more dependable.

Modern fine-tuning does not always require retraining every parameter. Parameter-efficient methods such as LoRA keep the base model largely frozen and train comparatively small adapter parameters instead. Hugging Face describes PEFT as adapting large pretrained models while updating only a small number of additional parameters, reducing memory and storage requirements.

Fine-tuning is powerful, but it is not a sensible substitute for a database. If your price list changes every Monday, do not retrain every Tuesday.

Tools: change what the AI can do

Tools turn an answering machine into an acting system.

Tools let the model operate on real systems. The pattern is straightforward:

Request → model selects tool → application executes tool → result returns → model answers

This distinction matters for security. The model should not receive unrestricted access to your computer merely because it can request an action. Your application defines the available tools, validates parameters and decides which actions require confirmation.

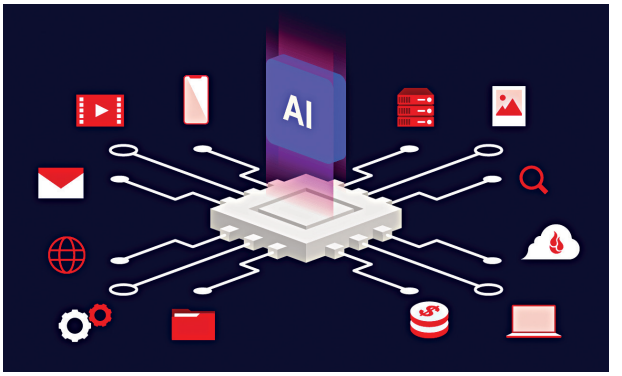
Tool ecosystems are also becoming more standardised. The July 2026 Model Context Protocol specification defines a host-client-server architecture through which AI applications can discover resources, prompts and tools from external services. MCP illustrates where AI systems are heading: the model becomes one reasoning component inside a larger network of controlled capabilities.

RAG, Fine-Tuning, Tools: Or all three?

One of the easiest ways to waste time is choosing an unnecessarily complicated solution. Use this rule before writing code:

If the model needs instructions, prompt it. If it needs information, retrieve it. If it needs repeatable behaviour, fine-tune it. If it needs to perform an external action, give it a tool.

Consider an AI assistant for an electronics retailer. It needs to answer questions using the latest return policy. That is a RAG problem because policies may change.



It must answer product-comparison queries using a house format containing price, strengths, weaknesses and recommendation. Start with prompting. If compliance remains poor across many examples, fine-tuning becomes worth considering.

It needs to tell a customer whether a laptop is currently in stock. That requires a tool connected to live inventory. If the business also has thousands of historical support conversations showing exactly how difficult warranty cases should be classified, that becomes potential fine-tuning data.

A mature application may therefore use all four techniques simultaneously. A system prompt establishes rules. Retrieval supplies current policies. An adapter teaches specialised behaviour. Tools perform live lookups. The skill lies in assigning each problem to the right layer.

Think pipelines, not chat windows

A chat box makes AI systems look deceptively simple because all the machinery is hidden behind a rectangle and a Send button. As a builder, start drawing the pipeline.

Take a private-document assistant. The visible interaction is:

"Can I claim a hotel bill from this trip?"

Behind that one question, the application might identify which document collection to search, retrieve relevant travel-policy passages, construct a prompt containing those passages, ask the model for an answer and verify that citations are present before displaying it.

A research assistant might retrieve reports, run calculations, ask the model to interpret the results and format a report.

Once you think this way, model choice becomes only one design decision among many. A smaller model with excellent retrieval and tightly designed tools can be more useful for a particular job than a larger model receiving poor context.